SWEATERCO

GROUP 11 / TRACK 2 / SPRINT 2 REVIEW

StitchFlow

The Quick-Response Subcontracting Desk

Say the order. StitchFlow finds the right workshop.

PRIMARY MINIMISE LATENESS

SECONDARY MINIMISE DEFECTS

Chen Zepeng (A0156784N)
Present (in person)

Duong Thanh Nhan (A0296326W)
Absent: business trip to China

Joshua Wesley Gloor (A0296944J)
Present (in person)

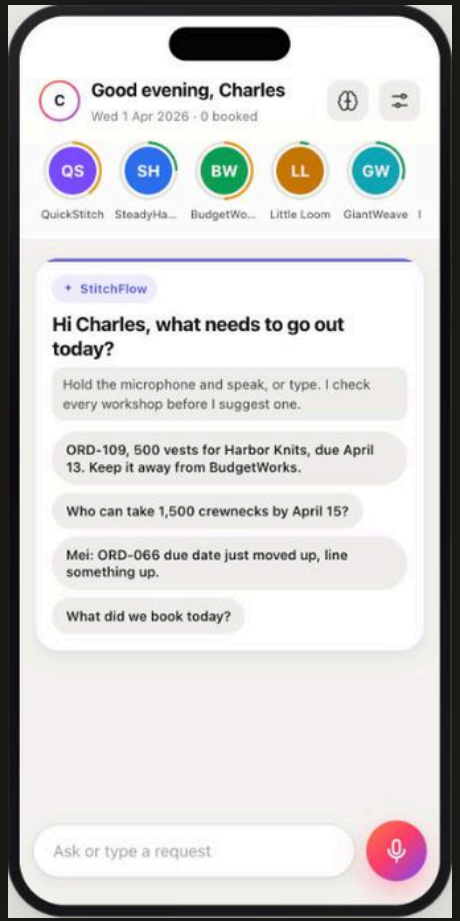
Sridhar Niyanthri Rajeswari (A0351151U)
Present (in person)

Sundara Rajan Saadhvika (A0352594W)
Present (in person)

Tang Zheling (A0352177B)
Present (in person)

GITHUB github.com/JoshuaGloor/dss5105-capstone-project

DEMO VIDEO [YouTube link: add after upload]



DSS5105 CAPSTONE / 30 OCTOBER 2026

The dispatcher decides on the factory floor, in seconds

THE USER

The production dispatcher at SweaterCo

On the floor with a phone, not at a desk. Requests arrive in a group chat, and most are messy: missing dates, vague orders, banned workshops.

A REAL REQUEST FROM THE INBOX

"ORD-109, 500 vests for Harbor Knits, due Apr 13. ...and keep it away from BudgetWorks, the last batch came back rough."

WHAT THE DISPATCHER NEEDS

1 A safe workshop, its arrival date and why, in seconds

2 The constraint honoured, and who asked recorded

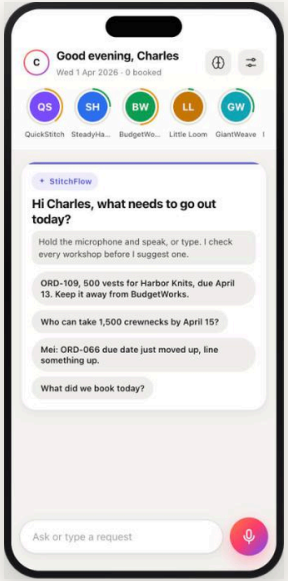
3 One deliberate step to commit; a record afterwards

WHAT CHANGED SINCE SPRINT 1

SPRINT 1	SPRINT 2
Earliest finish tied us on lateness	**Default beats it: 1.42% late vs 1.67%, 68% fewer late days**
Biggest capacity won on defects	**Defects setting wins: 4.90% defective pieces vs 5.28%**
Fairness not tuned	**Fairness setting: largest share 19.2% vs random 22.6%**
Typing at a desk only	**Voice on the phone, read aloud, slide to book**
No memory between requests	**Remembers confirmed constraints, and says when it applies one**

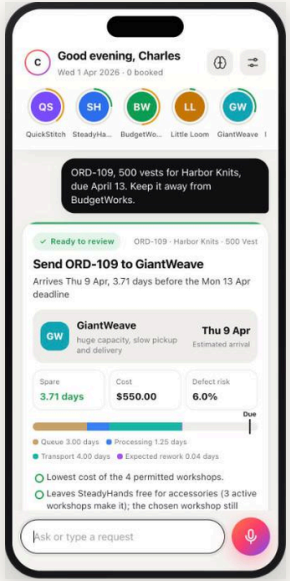
Course simulator, mean of 10 normal runs (seeds 5105-5114); presets chosen on development seeds first. docs/OBJECTIVE_STUDY.md

Say it, check it, slide to book, and it remembers



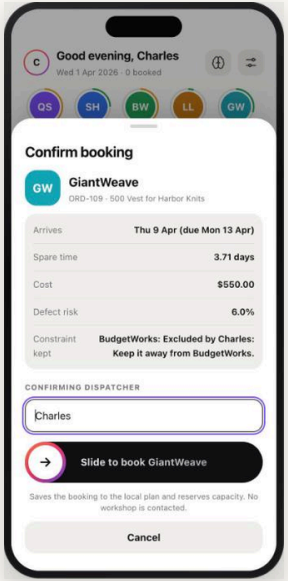
1 Ask

Hold the mic: order, date and any constraint, in plain words.



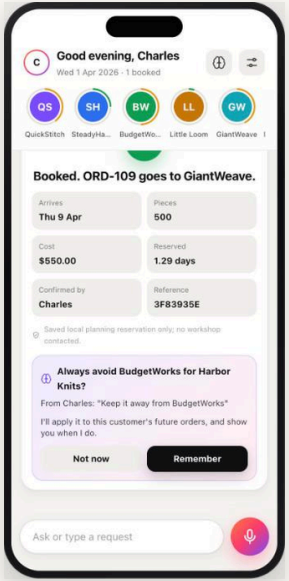
2 Answer

Best workshop, arrival and spare days, and why. Read aloud.



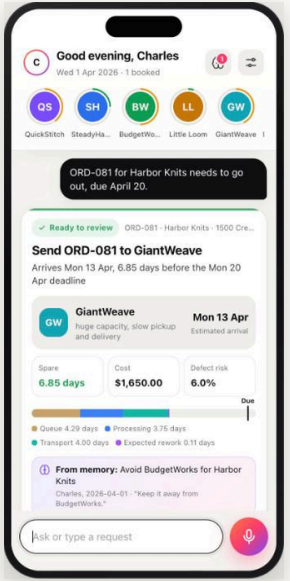
3 Confirm

"Book it" opens one sheet. One slide books it, with a name on it.



4 Remember

It offers to keep the constraint for this customer.

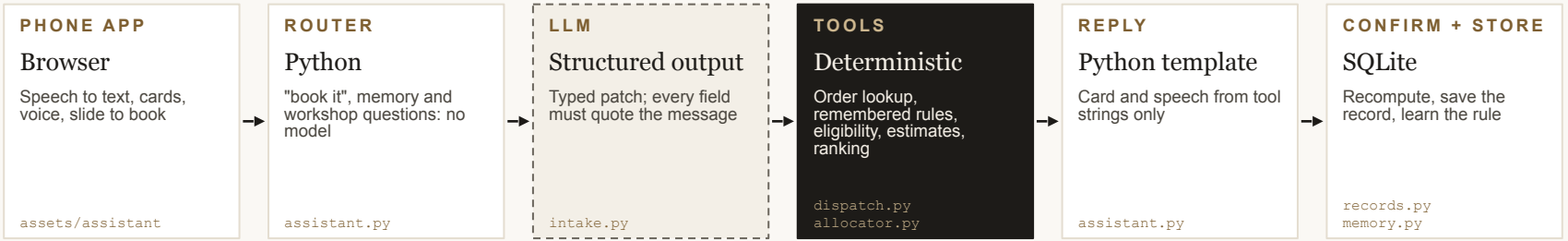


5 Reuse

Next order: the rule is applied, and the card says so.

Live screenshots of the running app (local plan, model: the one set in .env). Voice proposes; only the slide commits.

The model reads, Python computes, a person commits



Trust boundary: the model call is the only thing that leaves the machine. It never sees costs, queues or defect rates.

KEY CHOICES, AGAINST THE ALTERNATIVE WE CONSIDERED

WE CHOSE	OVER	BECAUSE
One workflow	A ReAct agent loop	Every request needs the same checks in order: testable, cheaper, no tool-choice errors
Browser speech recognition	Server speech-to-text	Free, live words while speaking, no extra key; typing always works
Python templates for replies	Model-written replies	No model arithmetic: dates and costs are copied from tools
Exact rules in SQLite	A vector-store memory	A constraint must apply exactly, with who said it, and be auditable
Slide to confirm	Book on a spoken "yes"	A misheard "yes" on a noisy floor cannot send work out

Every feature, its owner and its status

FEATURE	OWNER	STATUS
Voice assistant: phone UI, speech in and out, slide to book	Chen Zepeng	**Done**
Long-term memory: rules from confirmed bookings, log, profile	Chen Zepeng	**Done**
Four-objective scorer, frozen presets, objective lab	Chen Zepeng	**Done**
Capacity questions and workshop checks from chat	Chen Zepeng	**Done**
Chat-first desk redesign	Duong Thanh Nhan	**Done**
AWS deployment behind HTTPS (demo site)	Joshua Wesley Gloor	**Done**
Clarification reliability fixes and 247-assertion benchmark	Tang Zheling	**In progress**
Evaluation set: 50 new held-out requests, written and labelled	Saadhvika and Niyanthri	**In progress**
Independent labels for the 30 inbox requests	Saadhvika and Niyanthri	**In progress**
Live model run on the full evaluation set (30 inbox + 50 new)	Tang Zheling	**Planned**
Usability test: two people acting as dispatchers, four tasks	[TBD]	**Planned**

6 of 11 done. Code: github.com/JoshuaGloor/dss5105-capstone-project (PR #15 for this sprint, PR #14 for clarification reliability).

What we measure, against what, and the hard cases

WHAT WE MEASURE, AND WHY

Late batches and late days (our objective), defective pieces, largest share, change under a closure: the course simulator's own metrics. For language: the right behaviour on each request, and every number traced to a tool.

HOW THE EVALUATION SET IS BUILT

All 30 inbox requests, grouped by the 7 kinds in the data dictionary, plus 50 new held-out requests (rewordings, bait items) written and labelled by Saadhvika and Niyanthri. Labels are made separately, then agreed; a model judge counts only where it matches them.

BASELINE AND TRACK MINIMUM

Random, biggest capacity and cheapest, plus the earliest-finish rule. Minimum: beat all three baselines on late batches and late days, and handle all 30 requests by kind.

TODAY, COURSE SEEDS 5105-5114, NORMAL RUNS

OBJECTIVE	OURS	BEST REFERENCE	
Lateness (primary)	1.42% late, 0.057 late days	earliest finish 1.67%, 0.178	**Beats all**
Defects	4.90% defective pieces	biggest capacity 5.28%	**Beats all**
Fairness	largest share 19.2%	random 22.6%	**Beats random**
Robustness	+0.083 pts late under closure	random +4.000	**Steadier**

HARD CASES AND HOW WE HANDLE THEM

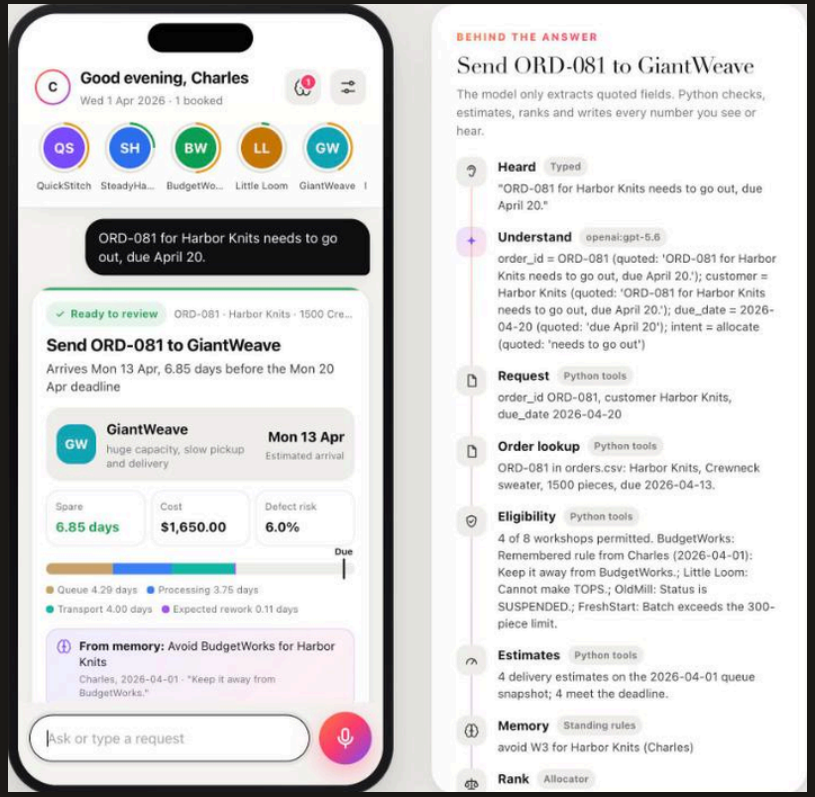
1. **Impossible date**
   Earliest date, days late, a two-shop split; no booking
2. **Workshop that cannot take it**
   Refused with the reason, plus the best alternative
3. **Misheard order on the floor**
   Order, customer, pieces read back from orders.csv; slide to commit

Presets chosen on development seeds 6000-6029; the course seeds were run twice, as disclosed in docs/OBJECTIVE_STUDY.md.

DEMO

Charles, as the dispatcher

1. Says a real inbox request out loud
2. Hears the best workshop and why
3. Says "book it" and slides to confirm
4. Lets it remember the constraint
5. A new order: the rule is applied
6. Hard case: a workshop that cannot take it, and an impossible date
7. Behind the answer: what the model read, what Python computed



STITCHFLOW

Thank you.

Questions are welcome.

GITHUB github.com/JoshuaGloor/dss5105-capstone-project

DEMO VIDEO [YouTube link: add after upload]

SAY THE ORDER / PYTHON CHECKS / A PERSON COMMITS